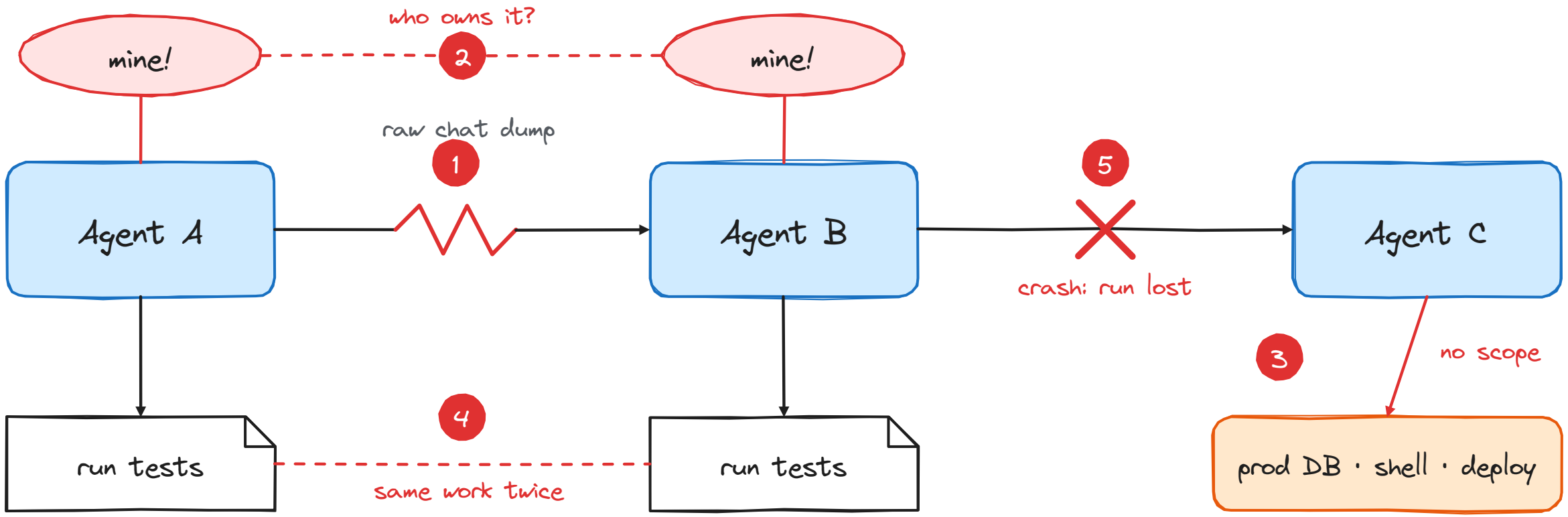
The Problem

PROPOSED DESIGN

Agents hand work to each other today with no shared contract



1 Context loss
decisions vanish in long chats

2 Unclear ownership
two agents think the task is theirs

3 Open permissions
tools shared with no scope

4 Duplicated work
no record of who did what

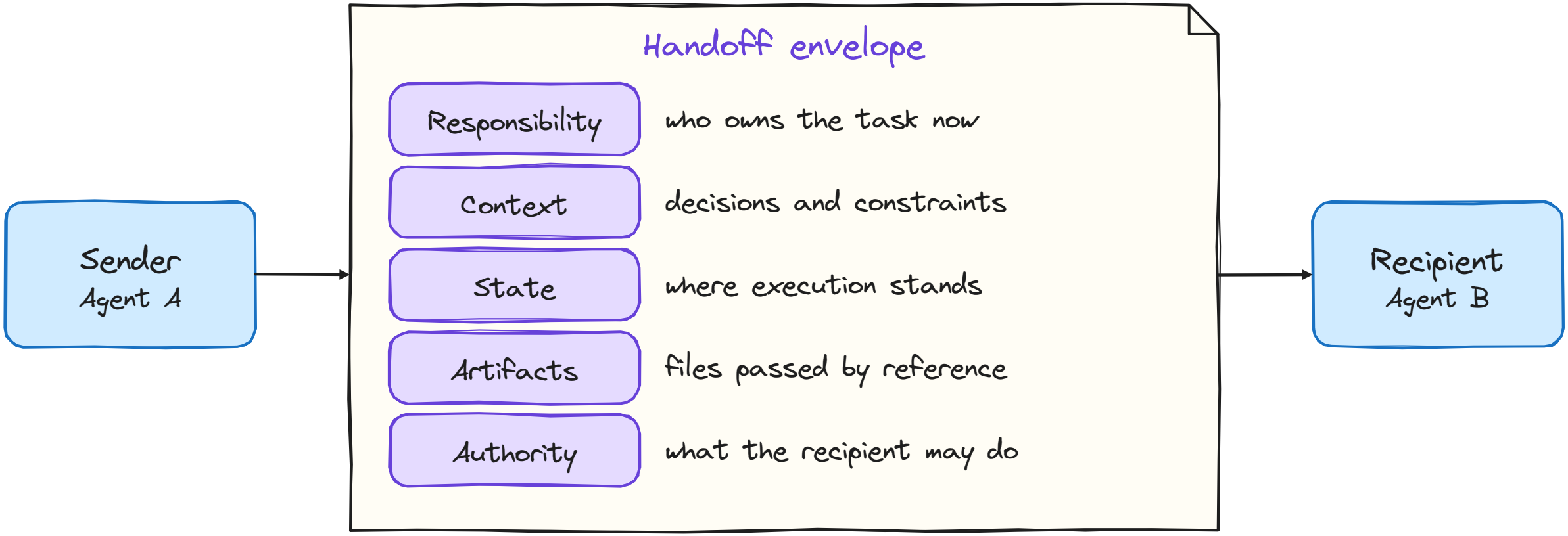
5 No recovery
a failure restarts everything

Agents Tools Problem / failure

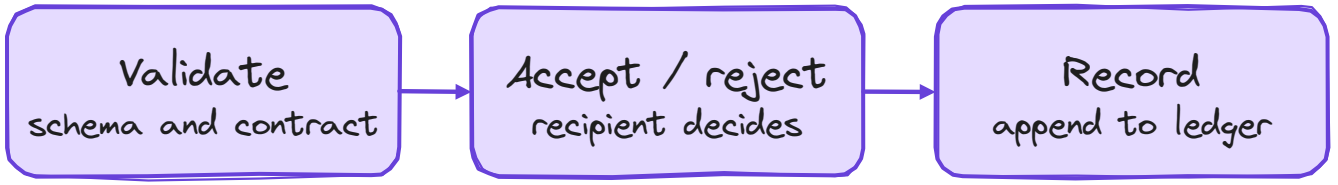
The Core Idea

PROPOSED DESIGN

AHP is one standard envelope for passing a task between agents



protocol rules around every handoff

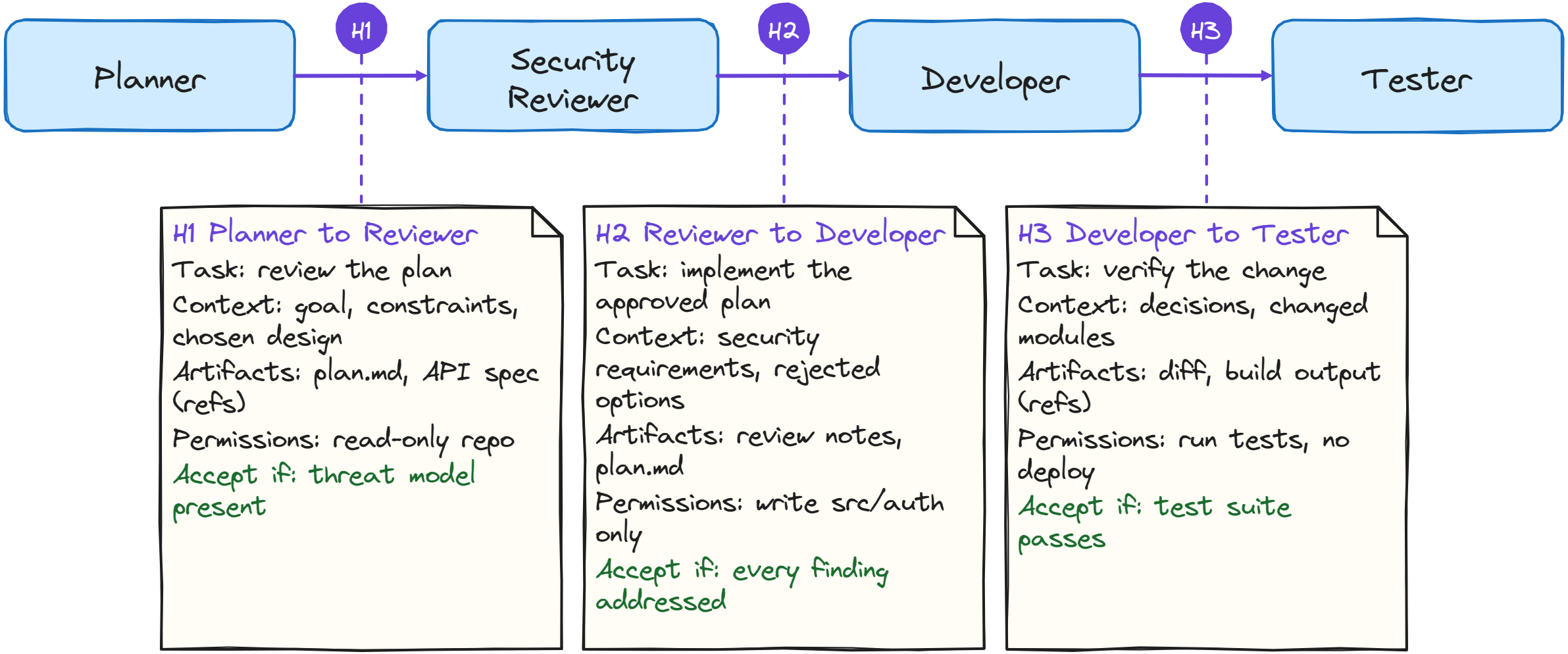


Agents Orchestration

A Real Example

PROPOSED DESIGN

Planner, Security Reviewer, Developer, Tester: each handoff moves task, context and authority

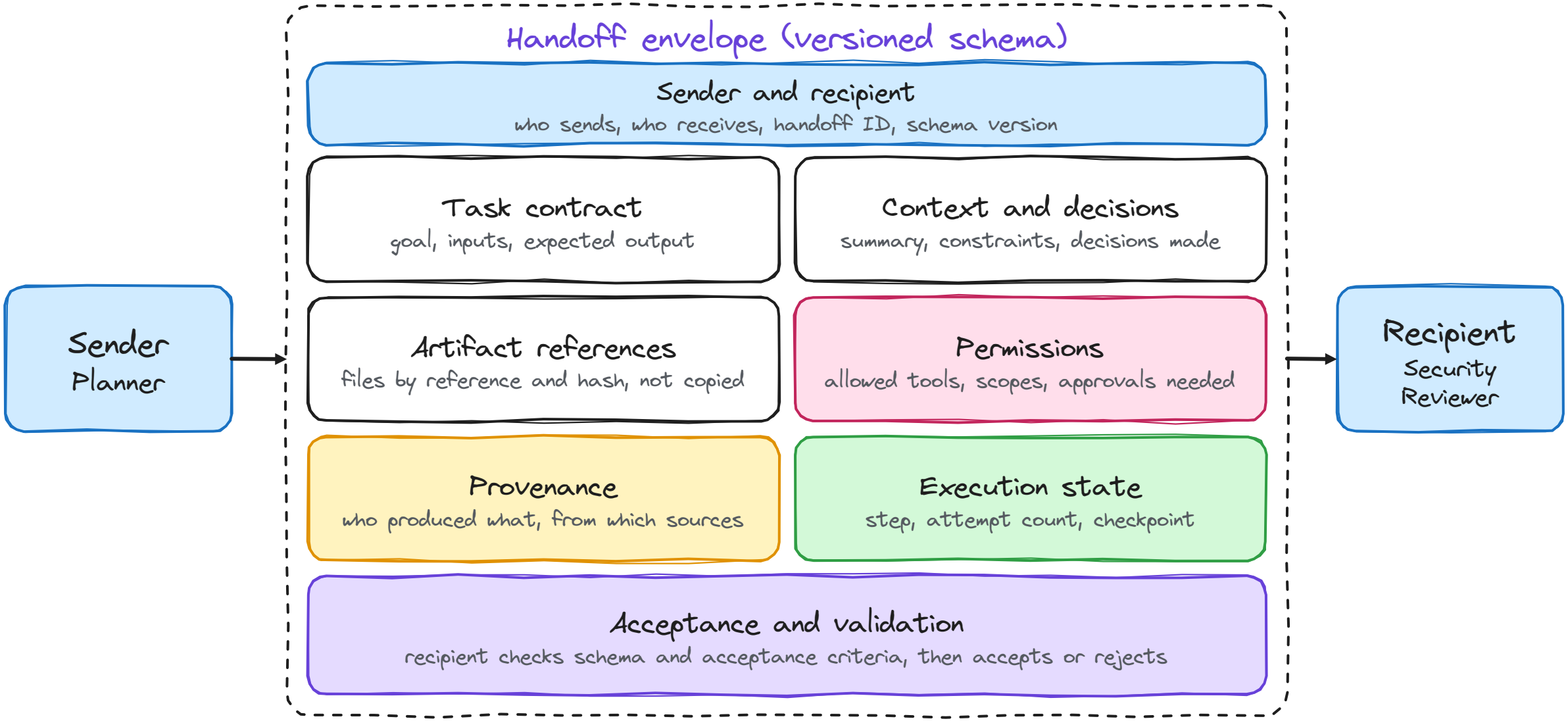


Responsibility, context and authority move together. Each agent gets only what its step needs.

Inside a Handoff

PROPOSED DESIGN

One versioned envelope carries everything the recipient needs

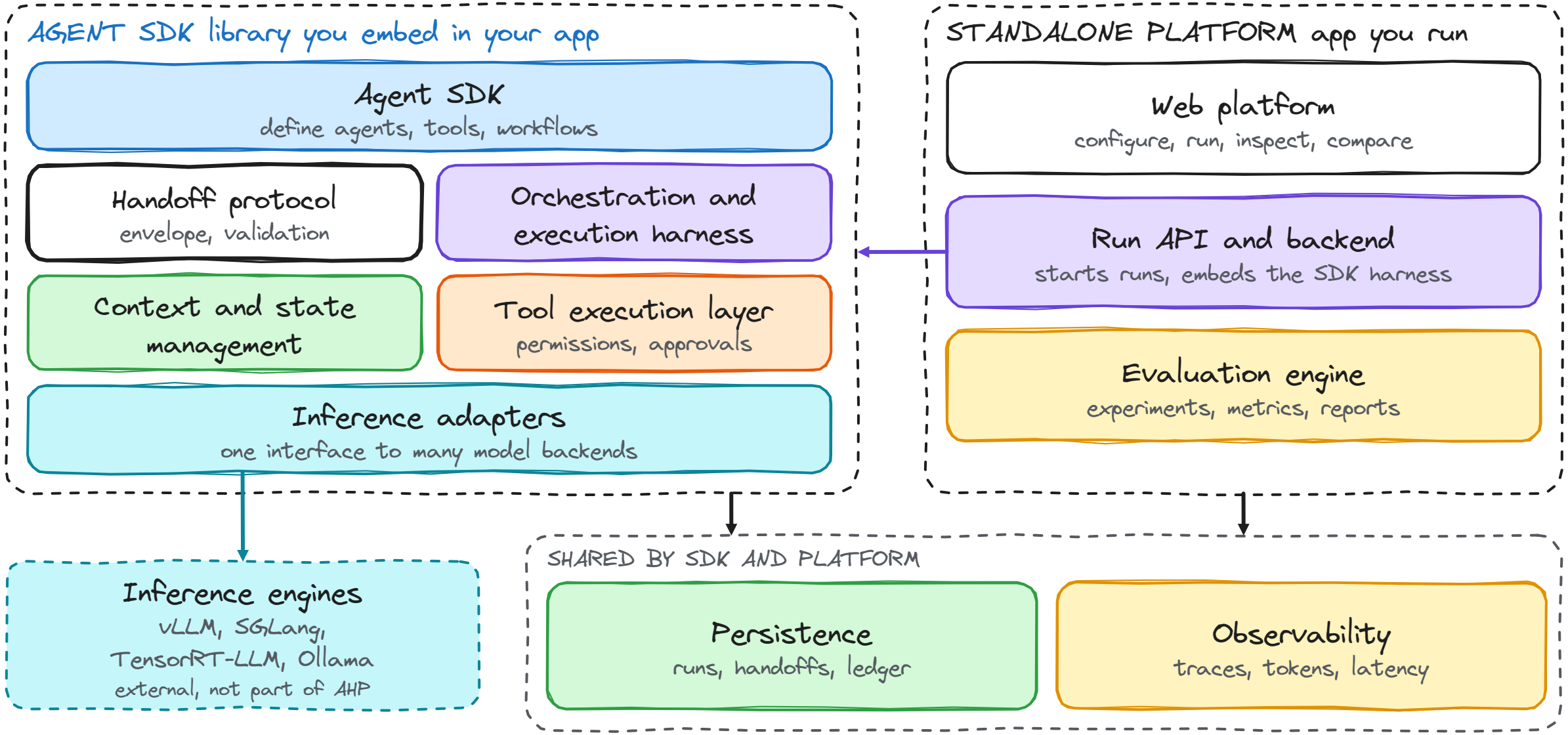


Agents · Orchestration · Storage / state · Evaluation / observability · Human approval

System Architecture

PROPOSED DESIGN

Two products, one core: a library you embed and an app you run

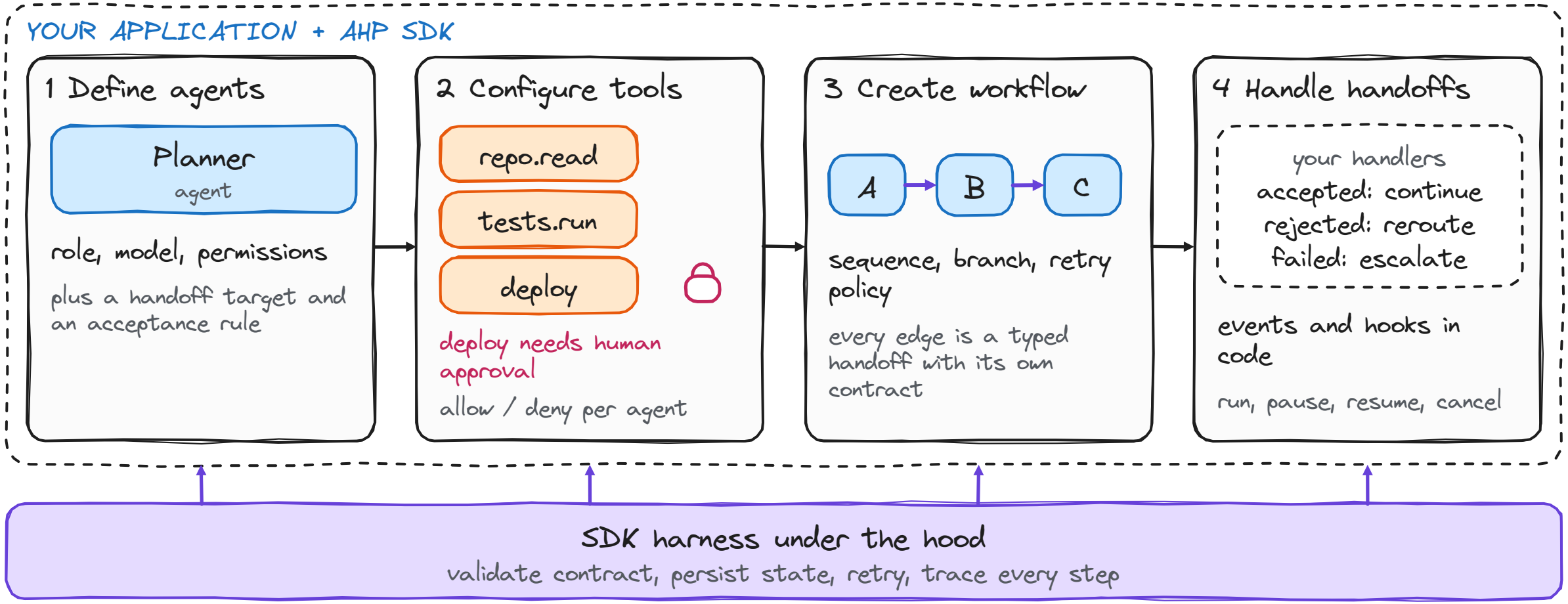


Agents · Orchestration · Tools · Storage / state · Inference engines · Evaluation / observability

SDK / Framework

PROPOSED DESIGN

Four steps from import to a working multi-agent workflow



Illustrative structure. API names and SDK language (Python, TypeScript or both) are still open decisions.

Agents Orchestration Tools Human approval

Standalone Platform

PROPOSED DESIGN

Configure, run, inspect, recover and compare without writing code

1 Configure agents

agents

Role — Planner

Model — engine + model

Tools — repo.read, tests.run

2 Connect steps

workflow

Plan → Review → Build

graph view of agents and handoffs

editor depth: open decision

3 Execute runs

runs

Planner — done

Reviewer — running

Developer — waiting

4 Inspect handoffs

run timeline

handoff envelope

5 Review failures

failures

Tester: tests failed

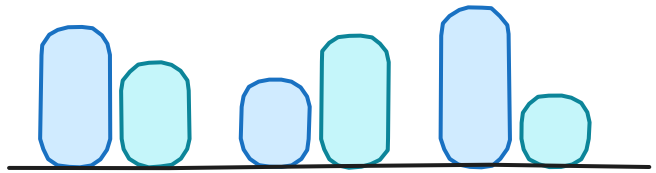
Reviewer: contract rejected

retry | resume | reassign

6 Compare results

experiments

run A run B sketch, no data

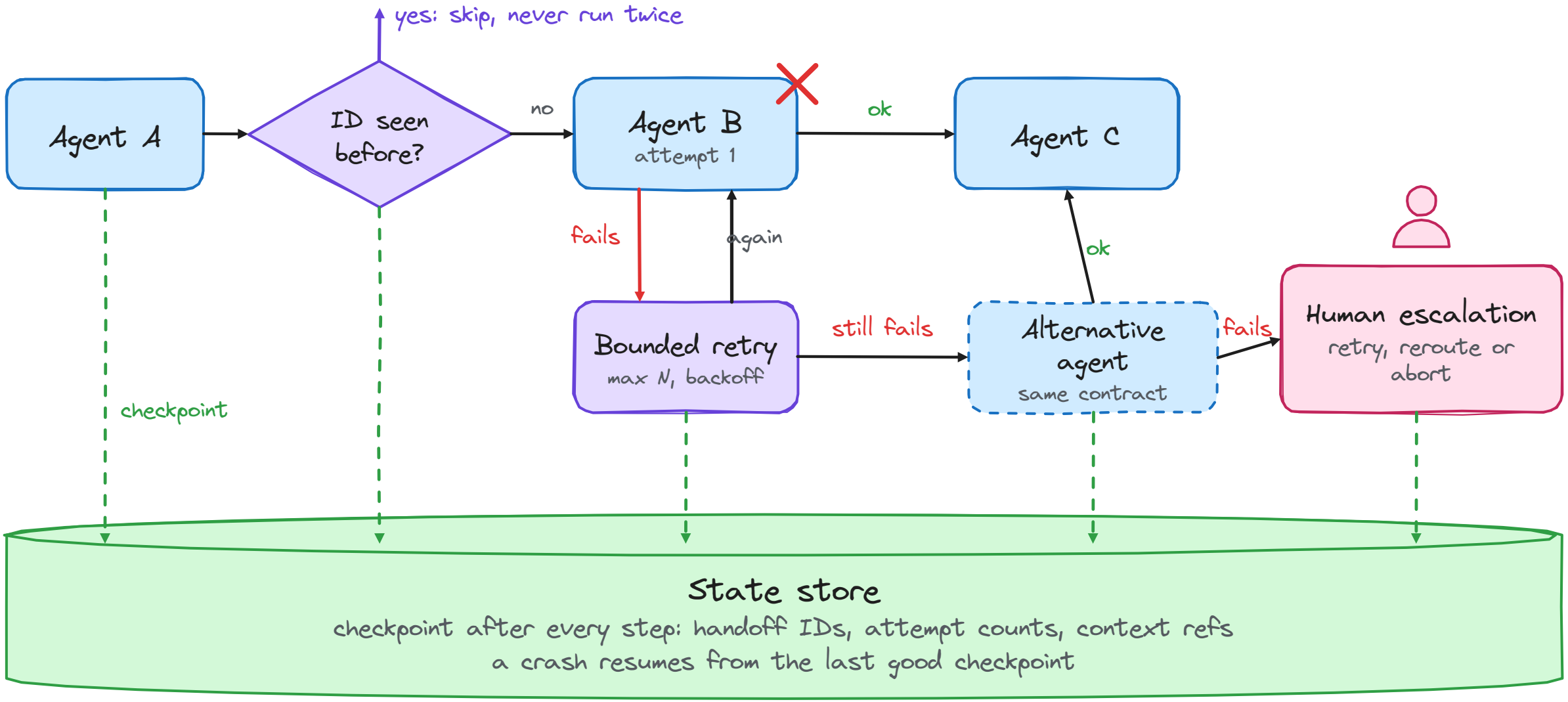


Agents | Orchestration | Evaluation / observability | Human approval

Reliability and Recovery

PROPOSED DESIGN

Bounded retry, reroute, escalate: nothing runs twice and nothing is lost

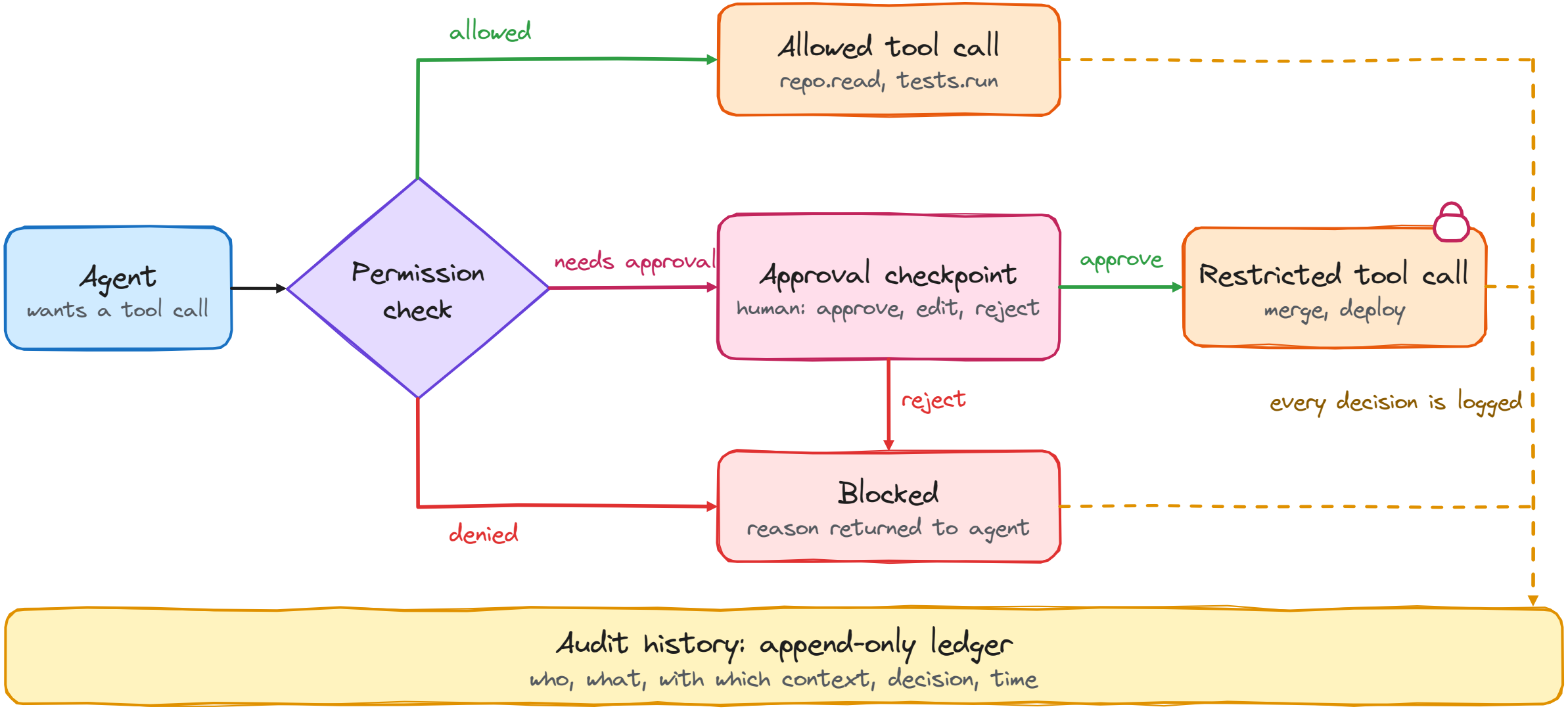


Agents | Orchestration | Storage / state | Human approval | Problem / failure

Security and Human Control

PROPOSED DESIGN

Every tool call is checked against policy; risky ones wait for a person; everything is logged

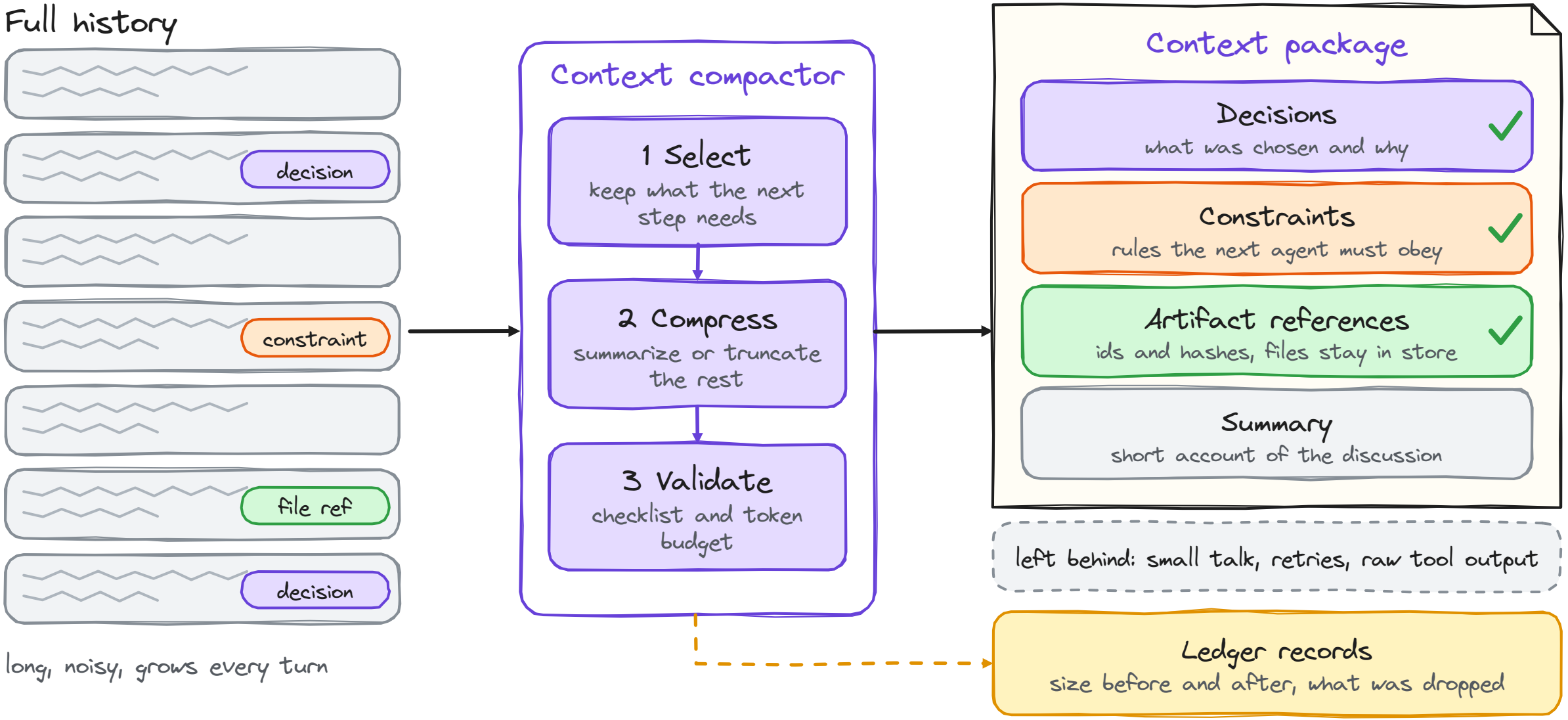


Agents | Orchestration | Tools | Human approval | Problem / failure | Evaluation / observability

Context Management

PROPOSED DESIGN

A long history becomes a small, validated package that keeps what matters

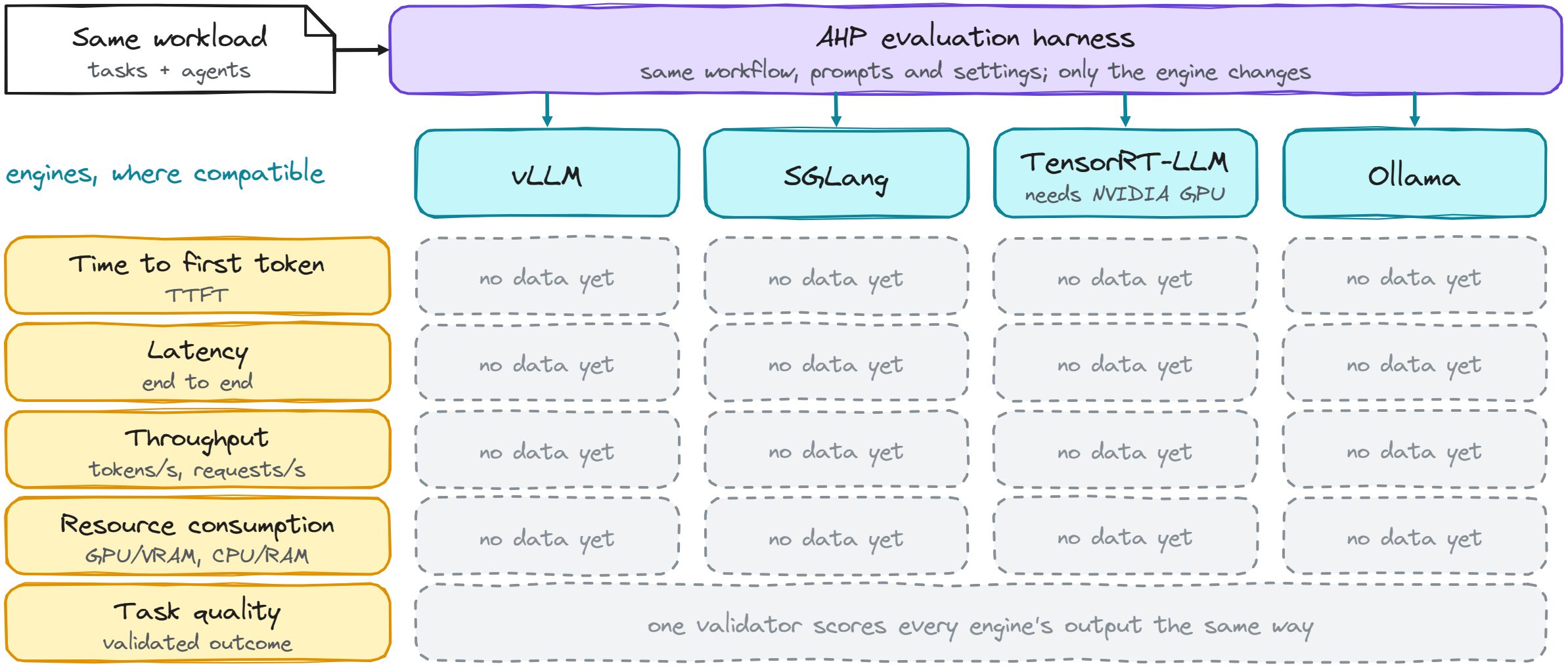


Orchestration · Tools · Storage / state · Evaluation / observability · Human approval

Inference Evaluation

PROPOSED DESIGN

The same agent workload, run on each engine the hardware allows, measured the same way



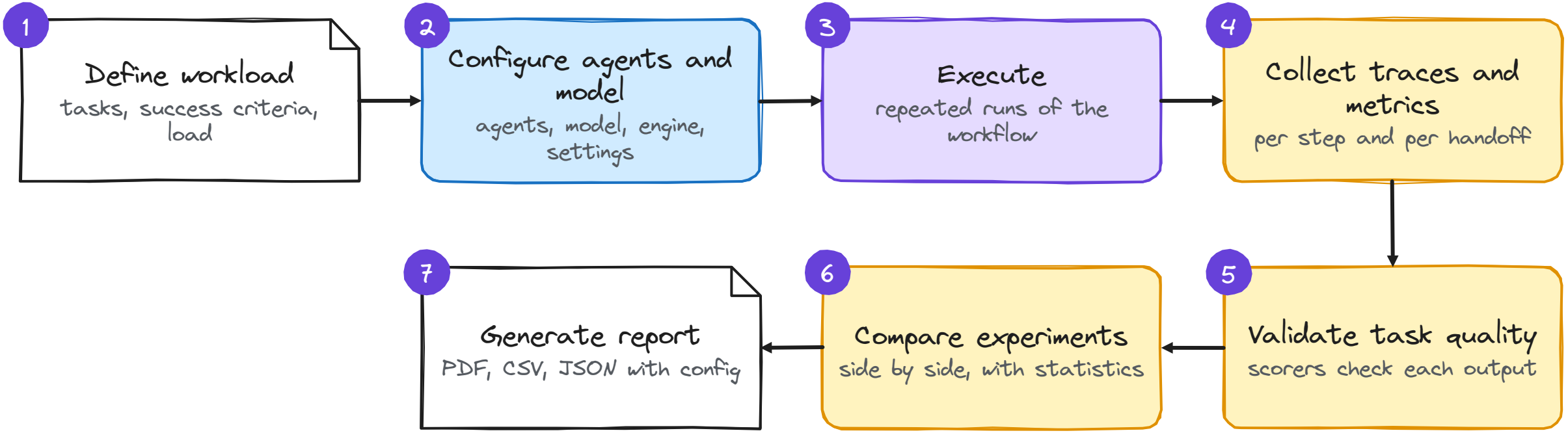
Empty on purpose: no results are shown. Values come from real runs on your hardware.

Orchestration Inference engines Evaluation / observability

Unified Evaluation Workflow

PROPOSED DESIGN

From a defined workload to a report, in one repeatable path



Stored with every experiment

- workload version
- agent + model config
- engine settings and seed
- environment and raw traces

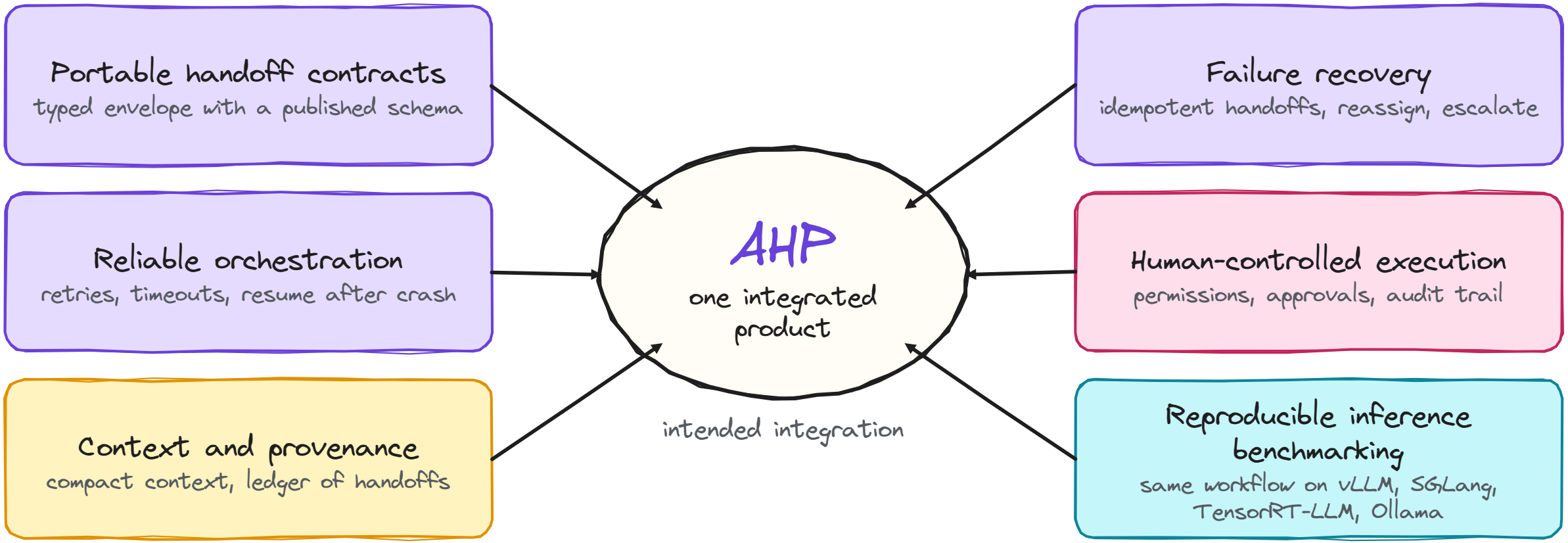
So any result can be traced back to exactly what produced it, and repeated.

Agents Orchestration Evaluation / observability

Competitive Differentiation

PROPOSED DESIGN

Six capabilities we intend to integrate in one product



Positioning hypothesis, not a uniqueness claim

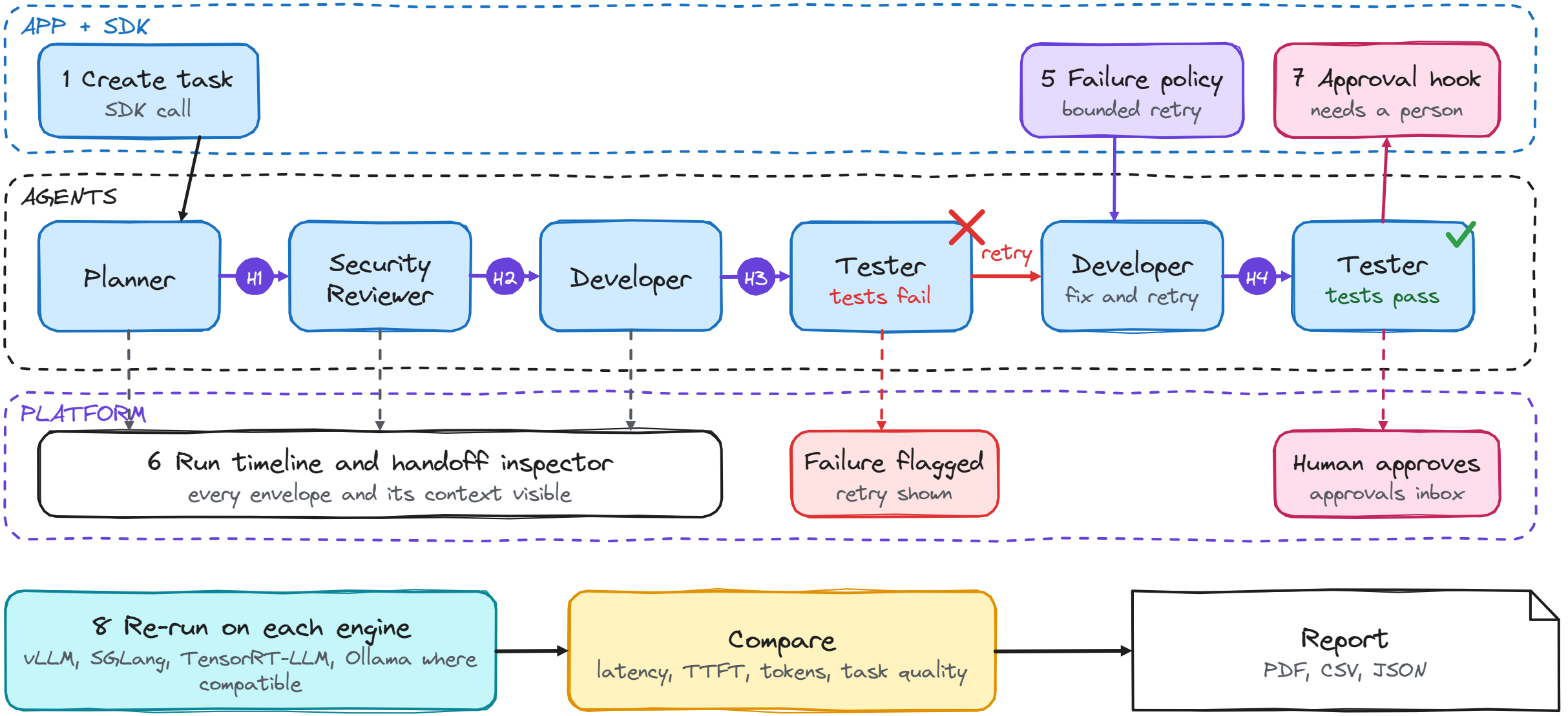
Our 2026-10-15 review found these capabilities spread across agent SDKs, platforms and benchmark tools, with many cells unverified. Hands-on checks come before any external claim.

Orchestration · Evaluation / observability · Human approval · Inference engines

End-to-End Demonstration

ILLUSTRATIVE SCENARIO

One task, add a password-reset endpoint: SDK and platform working together



Agents · Orchestration · Human approval · Problem / failure · Evaluation / observability

Conclusion

PROPOSED DESIGN

What AHP provides, and the problem it solves

Reliable, controlled, measurable agent handoffs

Context loss, unclear ownership, open permissions, duplicated work, no recovery: one protocol, one SDK, one platform

Handoff protocol and SDK

typed envelope
context, state, permissions
retry, recovery, approvals

Standalone platform

configure and run
inspect and review failures
approve and compare

Inference evaluation

same workload, many engines
TTFT, latency, throughput
resources and task quality

Shared foundation: persistence, observability, audit ledger

Open decisions

- Serve models, or evaluate external engines only?
- How deep should the workflow editor go?
- SDK language: Python, TypeScript or both?
- Reflection and simulation: core or examples?

Agents · Orchestration · Inference engines · Storage / state